endava

endava

Docker Advanced

Agenda

01 What is Docker?

02 What does Docker solve?

03 Docker Architecture

04 Images

05 Storage

06 Networking

07 Orchestration Basics

08 Demo

09 Q&A

endava

What is Docker?

↳ Applications Everywhere.

Chapter

01 Applications Everywhere

Docker is a platform to build, ship, and run applications in containers.

It helps developers package code, dependencies, and configuration in a consistent way.

https://docs.docker.com/get-started/overview

https://docs.docker.com/get-started/docker_cheatsheet.pdf

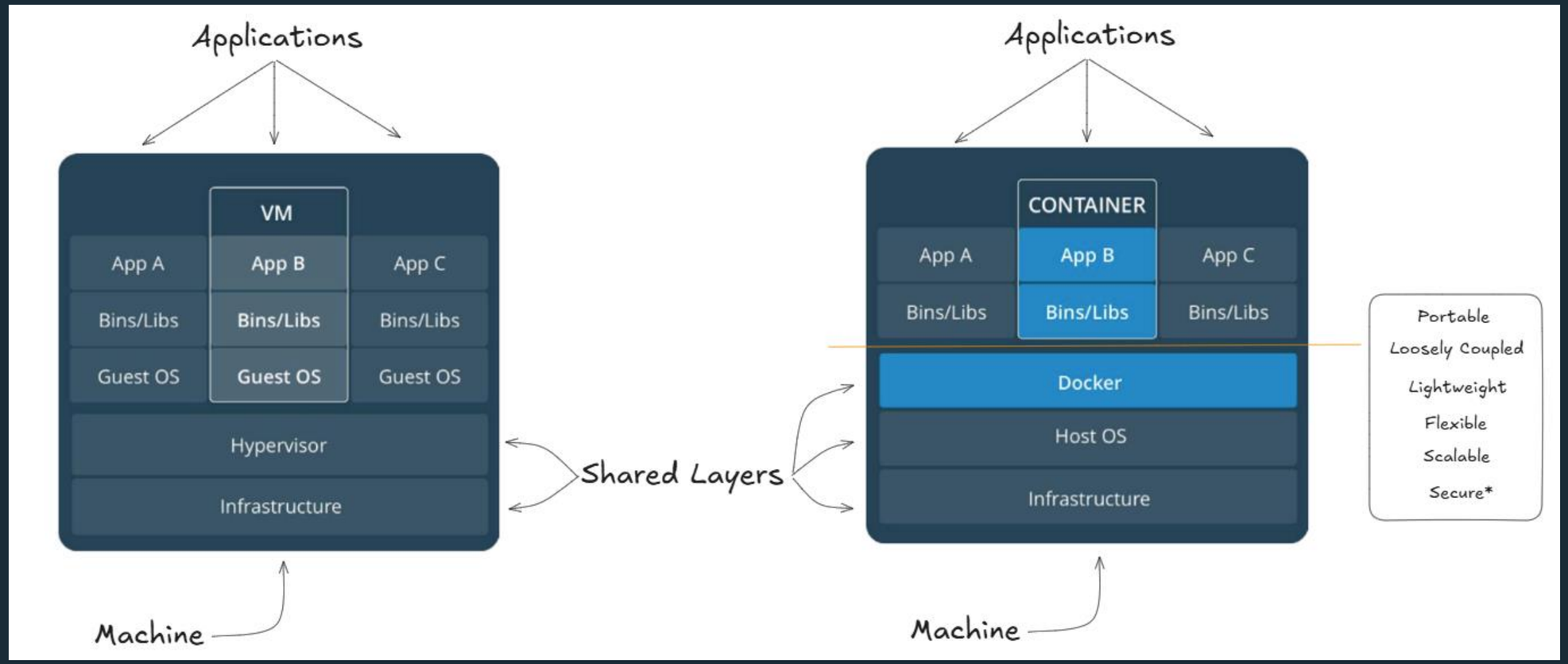
endava

What does Docker solve?

↳ Problem vs Solution.

Chapter

02 Problem vs Solution



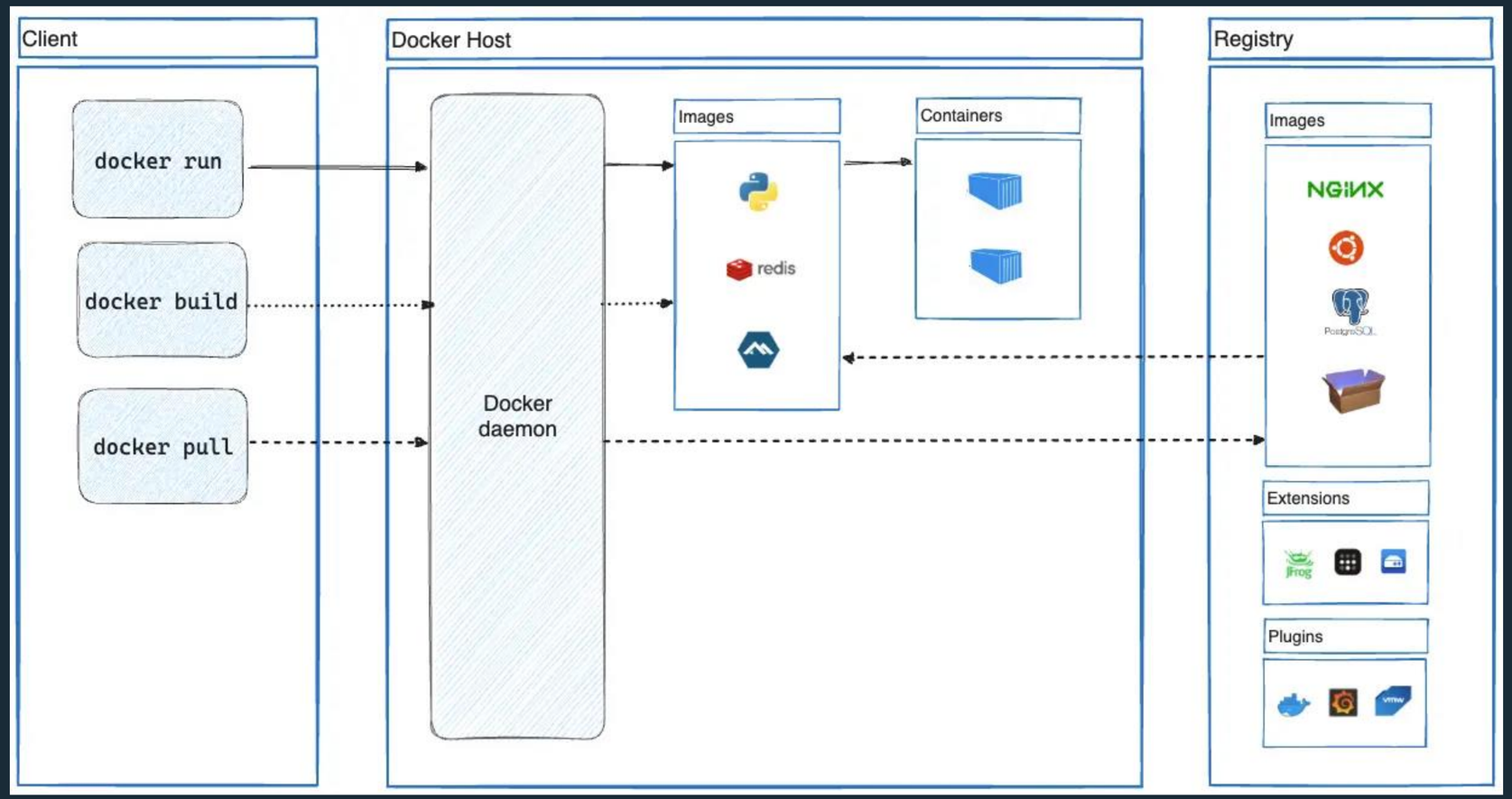
endava

Docker Architecture

Client vs Engine vs Registry.

Chapter

03 Client vs Engine vs Registry



https://docs.docker.com/get-started/overview

Client

Docker CLI

Is the primary way that many Docker users interact with Docker.

The docker command uses the Docker API. The Docker client can communicate with more than one daemon.

Daemon

dockerd

The Docker daemon listens for Docker API requests and manages Docker objects such as images, containers, networks, and volumes

Image & Container

Images

Template with instructions for creating a Docker container.

You might create your own images, or you might only use those created by others and published in a registry

Containers

A container is a runnable instance of an image. You can create, start, stop, move, or delete a container using the Docker API or CLI.

Registry

Registries

A registry stores Docker images. Docker Hub is a public registry that anyone can use. You can even run your own private registry.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

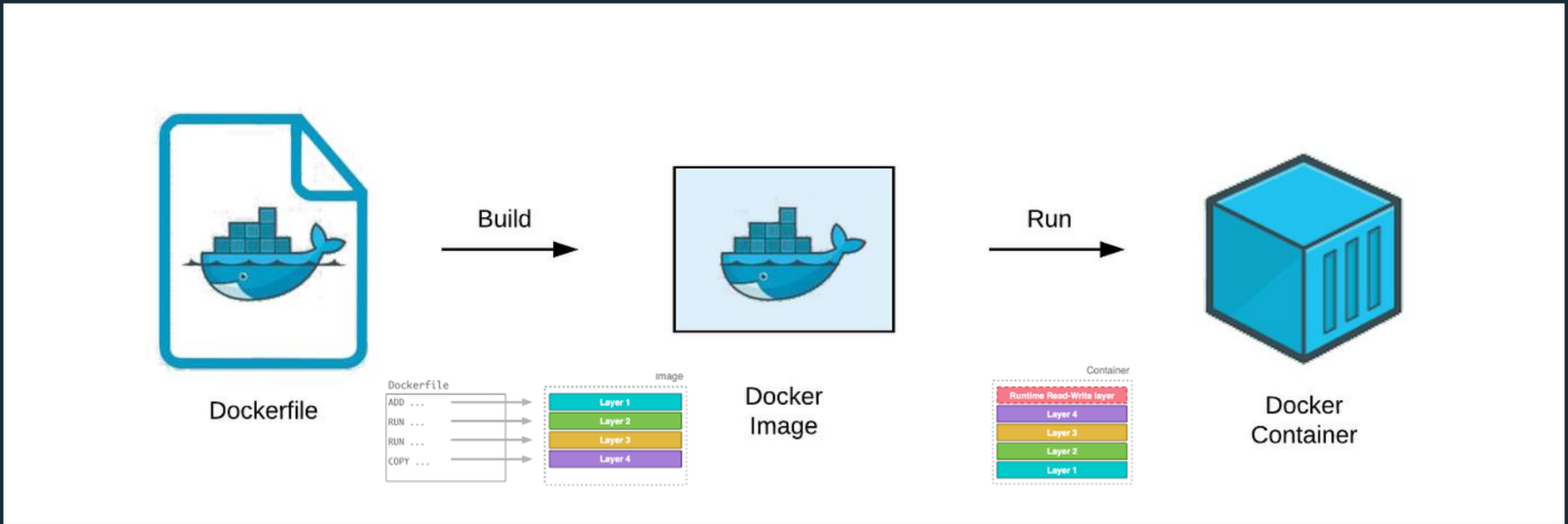
endava

Images

Manage, Build & Store.

Chapter

04 Manage, Build & Store



https://docs.docker.com/engine/reference/builder/

https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

```dockerfile
# ── BASE IMAGE ──────────────────────────────────────────────
FROM node:20-slim (last pushed 1 day ago)

# ── METADATA ────────────────────────────────────────────────
LABEL maintainer="your.name@endava.com"

# ── WORKING DIRECTORY ───────────────────────────────────────
WORKDIR /app

# ── DEPENDENCIES ────────────────────────────────────────────
COPY package*.json ./

# Install production dependencies only
RUN npm ci --omit=dev

# ── SOURCE CODE ─────────────────────────────────────────────
COPY . .

# ── SECURITY ────────────────────────────────────────────────
RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser
USER appuser

# ── PORT ────────────────────────────────────────────────────
EXPOSE 3000

# ── STARTUP ─────────────────────────────────────────────────
ENTRYPOINT ["node"]
CMD ["server.js"]
```

Instruction	Description
`ADD`	Add local or remote files and directories.
`ARG`	Use build-time variables.
`CMD`	Specify default commands.
`COPY`	Copy files and directories.
`ENTRYPOINT`	Specify default executable.
`ENV`	Set environment variables.
`EXPOSE`	Describe which ports your application is listening on.
`FROM`	Create a new build stage from a base image.
`HEALTHCHECK`	Check a container's health on startup.
`LABEL`	Add metadata to an image.
`MAINTAINER`	Specify the author of an image.
`ONBUILD`	Specify instructions for when the image is used in a build.
`RUN`	Execute build commands.
`SHELL`	Set the default shell of an image.
`STOPSIGNAL`	Specify the system call signal for exiting a container.
`USER`	Set user and group ID.
`VOLUME`	Create volume mounts.
`WORKDIR`	Change working directory.

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

Manage Local Images

→ Build Images

→ Store Images

Hands On!

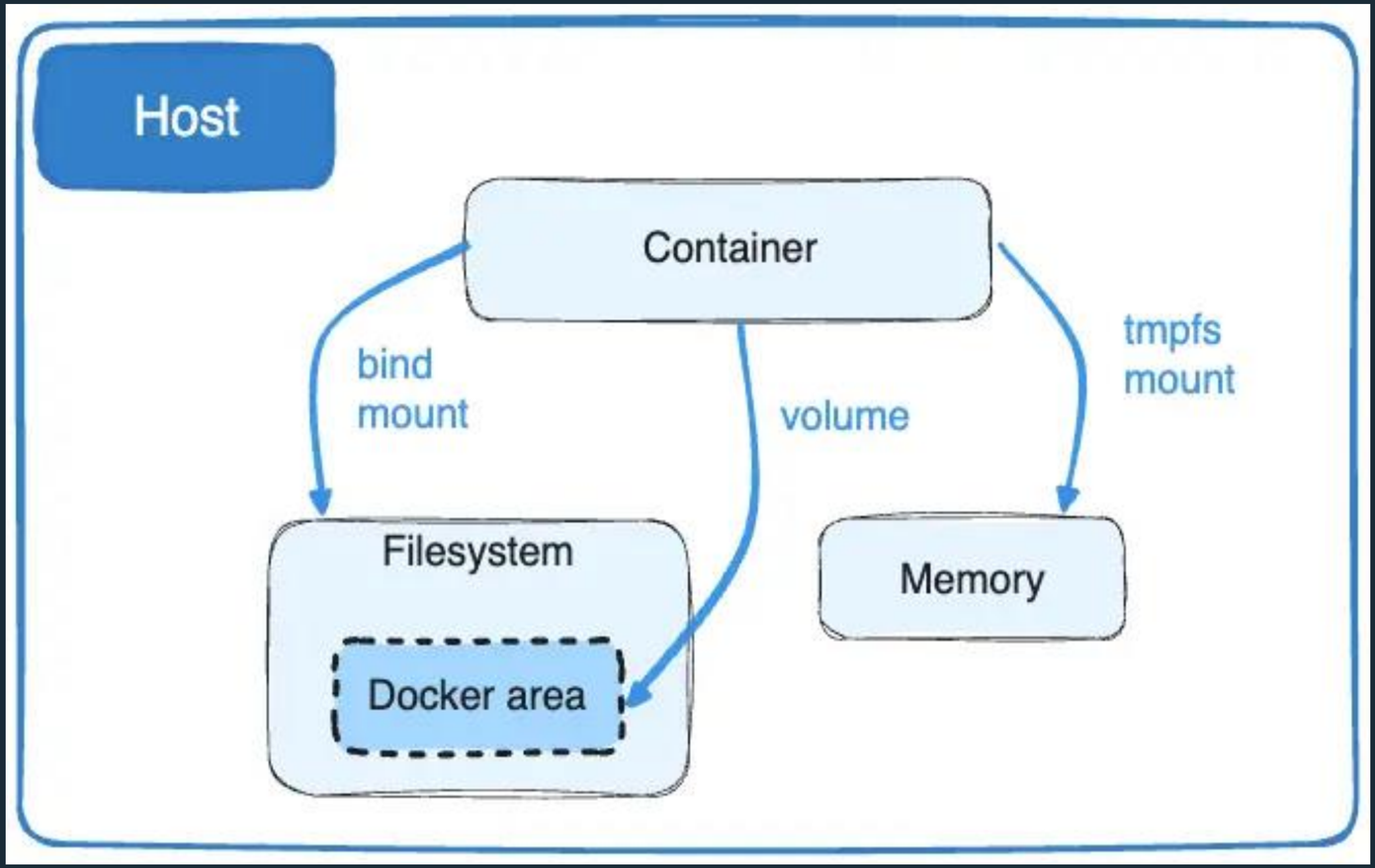
© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

endava

Runtime
Storage & Networking.

Chapter

05 Storage



Bind mount

Bind mounts may be stored anywhere on the host system. They may even be important system files or directories. Non-Docker processes on the Docker host or a Docker container can modify them at any time.

```
$ docker run -d \
  --name container \
  --mount type=bind,source="$(pwd)"/target,target=/app \
  image:latest
```

Volume

Stored in a part of the host filesystem which is *managed by Docker* (/var/lib/docker/volumes/ on Linux). Non-Docker processes should not modify this part of the filesystem. Volumes are the best way to persist data in Docker.

```
$ docker run -d \
  --name container \
  --mount source=vol,target=/app \
  image:latest
```

Tmpfs

Temporal filesystem mounts are stored in the host system's memory only and are never written to the host system's filesystem.

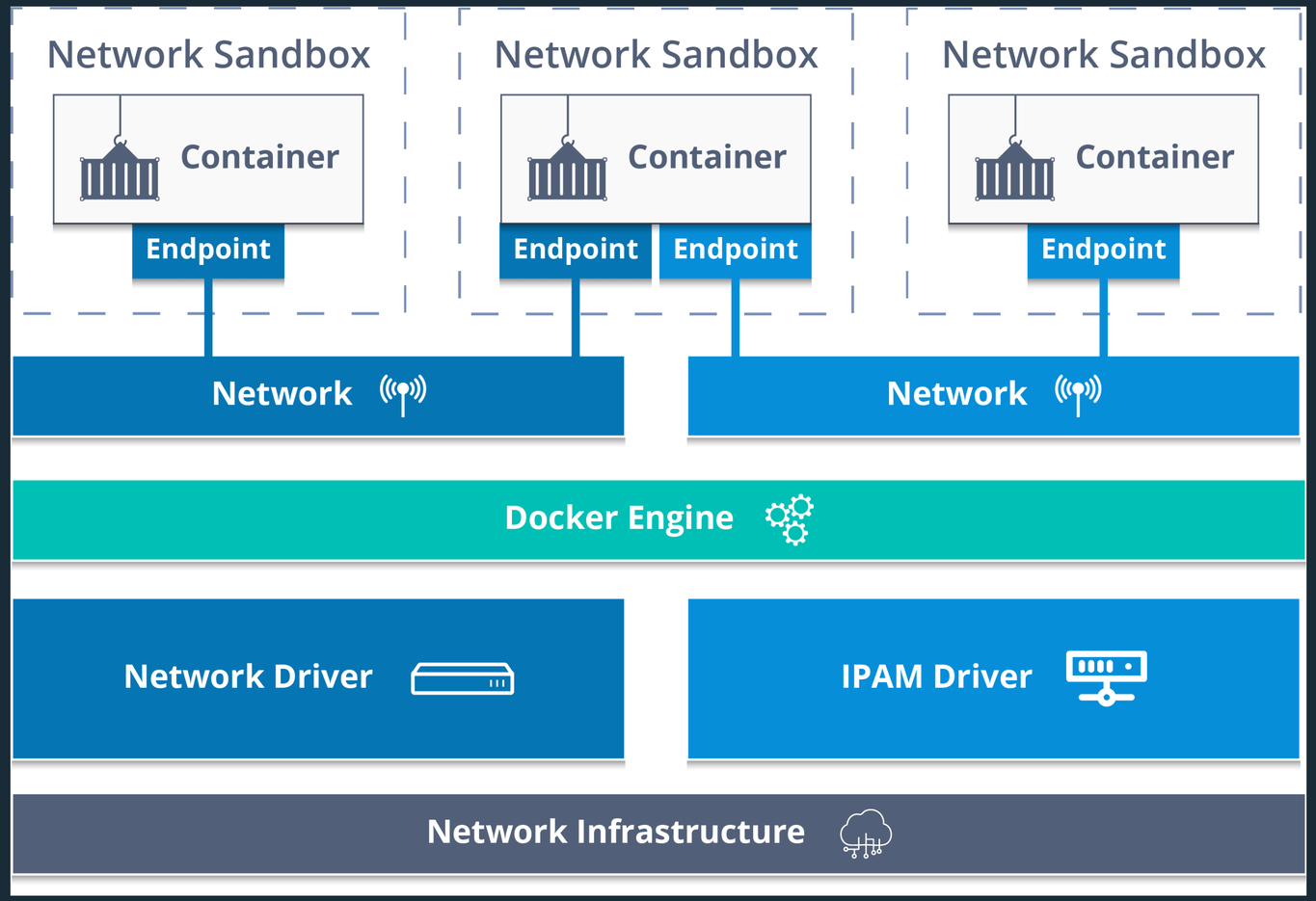
```
$ docker run -d \
  --name container \
  --mount type=tmpfs,destination=/app \
  image:latest
```

https://docs.docker.com/storage/

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

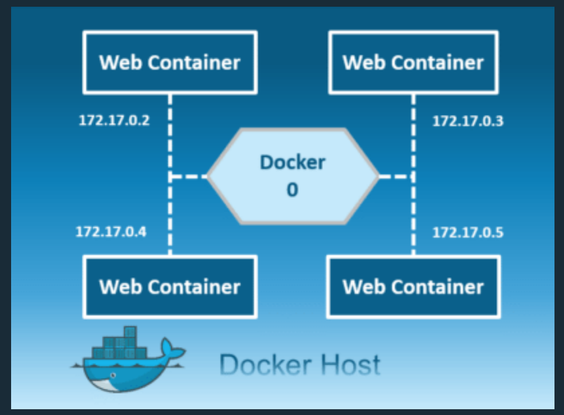
Chapter

06 Networking



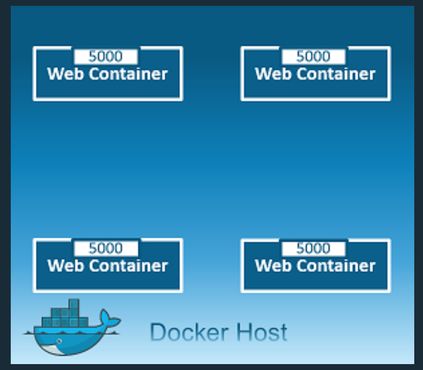
Bridge

Bridge networks apply to containers running on the same Docker daemon host and we need communicate them.



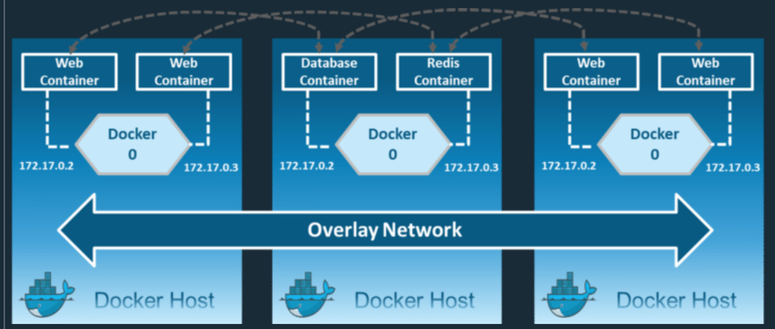
Host

With host networks, the container's network stack is not isolated from the Docker host. Container doesn't get its own IP-address allocated, uses the host's IP address.



Overlay

Overlay network driver creates a distributed network among multiple Docker daemon hosts. Overlay networks are often used to create a connection between Swarm services



https://docs.docker.com/network/

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

endava

Orchestration Basics

Docker compose.

07 Docker Compose

A tool for defining and running multi-container applications using a single YAML file.

Spin up multiple services with a single command.

Define services, networks, volumes, and environment variables in one place.

Reproduce development environments consistently on any machine.

https://docs.docker.com/reference/compose-file/

```yaml
version: '3.8'

services:

  # ── APP: Web Service ──────────────────────────────────────────
  app:
    build: ./app            # Build the image from the Dockerfile inside ./app
    container_name: web_service
    ports:
      - "5000:5000"         # Expose port 5000 to the host → localhost:5000
    environment:
      - REDIS_HOST=redis_service   # Use the service name as hostname (not localhost!)
      - REDIS_PORT=6379
    volumes:
      - ./app:/app          # Bind mount: local code syncs into the container
    networks:
      - my_network          # Both services must share the same network to talk

  # ── CACHE: Redis ──────────────────────────────────────────────
  redis:
    image: redis:alpine     # Pull official Redis image — no Dockerfile needed
    container_name: redis_service
    volumes:
      - redis_data:/data    # Named volume: data survives container restarts
    networks:
      - my_network

# ── VOLUMES ──────────────────────────────────────────────────────
# Docker manages this volume — data is NOT lost when the container stops
volumes:
  redis_data:

# ── NETWORKS ─────────────────────────────────────────────────────
# Custom bridge network: containers find each other by service name
# e.g. the app reaches Redis at redis_service:6379, not localhost:6379
networks:
  my_network:
    driver: bridge
```

© Copyright 2023 Endava • Confidential and Proprietary • Version 1.0

endava

Demo

↳ ... and obviously Q&A.

Thanks

Keep Learning

endava